1. Funkcje w Bashu i obsługa argumentów

- **Przekazywanie argumentów:** Funkcje w Bashu mogą przyjmować parametry w taki sam sposób jak skrypty.
- **Zmienne specjalne w funkcjach:**
  - `$#` – zwraca liczbę przekazanych argumentów.
  - `$@` – reprezentuje wszystkie przekazane argumenty jako listę.
- **Walidacja argumentów:** Wewnątrz funkcji można sprawdzać poprawność liczby argumentów (np. za pomocą instrukcji `if [ "$#" != 2 ]`) i w przypadku niezgodności natychmiast przerwać jej wykonywanie za pomocą polecenia `return`, co zapobiega dalszonemu, błędnemu przetwarzaniu.

2. Polecenie `let` do obliczeń arytmetycznych

- Polecenie `let` służy do wykonywania operacji matematycznych na **liczbach całkowitych**.
- Obsługuje bogaty zestaw operatorów uszeregowanych według priorytetów (m.in. inkrementacja/dekrementacja, potęgowanie `**`, operacje arytmetyczne, bitowe, logiczne oraz operatory przypisania).
- **Zasady działania:** Dzielenie przez zero jest wykrywane jako błąd. Status wyjściowy polecenia zależy od wyniku ostatniego argumentu (jeśli wynosi 0, `let` zwraca kod `1`, w pozostałych przypadkach `0`).

3. Wyrażenia arytmetyczne i praca z ułamkami

- **Liczby całkowite:** Standardowe mechanizmy Basha, takie jak `$((wyrażenie))` lub `$[wyrażenie]`, wykonują operacje wyłącznie na liczbach całkowitych (np. dzielenie `2/5` da w wyniku `0`, a nie `0.4`).
- **Potęgowanie:** Do potęgowania można używać operatora `**` (np. `$A=$[B**2]`).
- **Obsługa ułamków (`bc`):** Ponieważ Bash domyślnie nie obsługuje liczb zmiennoprzecinkowych, do obliczeń wymagających precyzji ułamkowej stosuje się zewnętrzny kalkulator **`bc`** w połączeniu z parametrem `scale` określającym liczbę miejsc po przecinku:

  Bash

  ```
  echo "scale=5 ; ($B/$C)" | bc
  ```